

```
// worker.js
// Единый Cloudflare Worker: раздаёт статический
сайт (index.html) и
// обрабатывает запросы к /api (вход, CRUD
списка, управление аккаунтами).
// Хранилище: Cloudflare KV (привязка PEOPLE_KV,
настраивается в дашборде,
// вкладка "Bindings").

const SUPER_LOGIN = 'St_admin1';
const SUPER_PASSWORD = 'st1FghXoL';

function corsHeaders() {
  return {
    'Access-Control-Allow-Origin': '*',
    'Access-Control-Allow-Headers': 'Content-
Type, X-Auth-Token',
    'Access-Control-Allow-Methods': 'GET, POST,
OPTIONS',
    'Content-Type': 'application/json'
  };
}

function json(data, status = 200) {
  return new Response(JSON.stringify(data), {
    status, headers: corsHeaders() });
}

async function sha256Hex(text) {
  const data = new TextEncoder().encode(text);
  const hash = await crypto.subtle.digest('SHA-
256', data);
  return [...new Uint8Array(hash)].map((b) =>
b.toString(16).padStart(2, '0')).join('');
```

```

}

async function hmacHex(secret, text) {
  const key = await crypto.subtle.importKey(
    'raw',
    new TextEncoder().encode(secret),
    { name: 'HMAC', hash: 'SHA-256' },
    false,
    ['sign']
  );
  const sig = await crypto.subtle.sign('HMAC',
    key, new TextEncoder().encode(text));
  return [...new Uint8Array(sig)].map((b) =>
    b.toString(16).padStart(2, '0')).join('');
}

function b64url(str) {
  return btoa(str).replace(/\+/g, '-')
    .replace(/\//g, '_').replace(/>=+$/g, '');
}
function fromB64url(str) {
  str = str.replace(/-/g, '+').replace(/_/g,
    '/');
  while (str.length % 4) str += '=';
  return atob(str);
}

async function signToken(secret, payloadObj) {
  const payload =
    b64url(JSON.stringify(payloadObj));
  const sig = await hmacHex(secret, payload);
  return payload + '.' + sig;
}

async function verifyToken(secret, token) {
  if (!token || token.indexOf('.') === -1)
    return null;

```

```

    const [payload, sig] = token.split('.');
    const expected = await hmacHex(secret,
payload);
    if (sig !== expected) return null;
    try {
        return JSON.parse(fromB64url(payload));
    } catch (e) {
        return null;
    }
}

async function hashPassword(secret, login,
password) {
    return sha256Hex(secret + ':' + login + ':' +
password);
}

function generateLinkCode() {
    const bytes = crypto.getRandomValues(new
Uint8Array(6));
    return [...bytes].map((b) =>
b.toString(16).padStart(2, '0')).join('');
}

async function sendTelegramMessage(env, chatId,
text, threadId) {
    const token = env.TELEGRAM_BOT_TOKEN;
    if (!token) return { ok: false, description:
'TELEGRAM_BOT_TOKEN не задан' };
    const payload = { chat_id: chatId, text };
    if (threadId) payload.message_thread_id =
threadId;
    const res = await
fetch(`https://api.telegram.org/bot${token}/send
Message`, {
        method: 'POST',
        headers: { 'Content-Type':

```

```

'application/json' },
  body: JSON.stringify(payload)
});
let data = {};
try { data = await res.json(); } catch (e) {}
return {
  ok: res.ok && data.ok,
  description: data.description || (res.ok ?
'' : `HTTP ${res.status}`),
  messageId: data.result &&
data.result.message_id
};
}

function sortPeople(list) {
  return [...list].sort((a, b) => {
    const aHas = a.post ? 1 : 0;
    const bHas = b.post ? 1 : 0;
    return bHas - aHas;
  });
}

async function handleTelegramWebhook(request,
env) {
  const kv = env.PEOPLE_KV;
  if (!kv) return json({ ok: true });

  const secretHeader = request.headers.get('X-
Telegram-Bot-API-Secret-Token');
  if (env.TELEGRAM_WEBHOOK_SECRET &&
secretHeader !== env.TELEGRAM_WEBHOOK_SECRET) {
    return json({ ok: false }, 401);
  }

  let update;
  try {
    update = await request.json();

```

```
    } catch (e) {
      return json({ ok: true });
    }

    const message = update.message;
    if (!message || !message.text ||
!message.chat) return json({ ok: true });

    const chatId = message.chat.id;
    const text = message.text.trim();

    if (text.startsWith('/start')) {
      const parts = text.split(' ');
      const code = parts[1];

      if (!code) {
        await sendTelegramMessage(env, chatId,
'Привет! Чтобы получать уведомления, откройте
сайт «Реестр участников», войдите под своим
логинем и нажмите «Привязать Telegram».');
        return json({ ok: true });
      }

      const pending = JSON.parse((await
kv.get('telegram_pending')) || '{}');
      const record = pending[code];

      if (!record || record.expires < Date.now())
{
        await sendTelegramMessage(env, chatId,
'Код недействителен или устарел. Сгенерируйте
новую ссылку на сайте и попробуйте снова.');
```

```

    bindings[record.login] = chatId;
    await kv.put('telegram_bindings',
JSON.stringify(bindings));

    delete pending[code];
    await kv.put('telegram_pending',
JSON.stringify(pending));

    await sendTelegramMessage(env, chatId, `✅
Telegram привязан к аккаунту «${record.login}».
Уведомления о непринятых заявках будут приходить
сюда 4 раза в день (10:00, 12:00, 16:00 и 20:00
МСК).`);
    return json({ ok: true });
  }

  if (text.startsWith('/setgroup')) {
    const chatType = message.chat.type;
    if (chatType !== 'group' && chatType !==
'supergroup') {
      await sendTelegramMessage(env, chatId,
'Эту команду нужно отправлять внутри группы,
которую вы хотите использовать для
уведомлений.');
```

return json({ ok: true });

```

    }

    const senderId = message.from &&
message.from.id;
    const bindings = JSON.parse((await
kv.get('telegram_bindings')) || '{}');
    const isLinkedEmployee = senderId &&
Object.values(bindings).some((v) => String(v)
=== String(senderId));

    if (!isLinkedEmployee) {
      await sendTelegramMessage(env, chatId, '❌

```

```
Только сотрудник с привязанным на сайте Telegram  
может назначить эту группу для уведомлений.');
```

```
    return json({ ok: true });  
  }
```

```
    await kv.put('notify_group_chat_id',  
String(chatId));  
    if (message.message_thread_id) {  
      await kv.put('notify_group_thread_id',  
String(message.message_thread_id));  
    } else {  
      await kv.delete('notify_group_thread_id');  
    }  
    await kv.delete('group_status_message_id');  
    await sendTelegramMessage(env, chatId, '✅  
Эта группа (раздел) назначена для живых  
уведомлений о непринятых анкетах. Убедитесь, что  
у бота есть право удалять сообщения (нужно для  
обновления списка).',  
message.message_thread_id);  
    await syncGroupStatus(env);  
    return json({ ok: true });  
  }
```

```
async function handleScheduled(env) {  
  const kv = env.PEOPLE_KV;  
  if (!kv) return;  
  
  const list = sortPeople(JSON.parse((await  
kv.get('people')) || '[]'));  
  const text = buildStatusText(list);  
  if (!text) return;  
  
  const bindings = JSON.parse((await  
kv.get('telegram_bindings')) || '{}');  
  const chatIds = Object.values(bindings);  
  for (const chatId of chatIds) {
```

```

        await sendTelegramMessage(env, chatId,
text);
    }
}

async function deleteTelegramMessage(env,
chatId, messageId) {
    const token = env.TELEGRAM_BOT_TOKEN;
    if (!token || !messageId) return;
    await
fetch(`https://api.telegram.org/bot${token}/dele
teMessage`, {
    method: 'POST',
    headers: { 'Content-Type':
'application/json' },
    body: JSON.stringify({ chat_id: chatId,
message_id: messageId })
});
}

function buildStatusText(list) {
    const notAccepted = list
        .map((p, i) => ({ p, num: i + 1 }))
        .filter((entry) => !entry.p.accepted);
    if (notAccepted.length === 0) return null;
    const lines = notAccepted.map(({ p, num }) =>
`№${num}. Ник: ${p.nick || '-'} | Юз: ${p.uz
|| '-'} | ДС: ${p.ds || '-'} | Принял:
${p.addedBy || '-'}`
    );
    return `⚠ Не приняты в клан
(${notAccepted.length}): \n\n${lines.join('\n')}`
;
}

async function syncGroupStatus(env) {
    const kv = env.PEOPLE_KV;

```

```

    if (!kv) return;

    const groupChatId = await
kv.get('notify_group_chat_id');
    if (!groupChatId) return;
    const threadId = await
kv.get('notify_group_thread_id');

    const oldMessageId = await
kv.get('group_status_message_id');
    if (oldMessageId) {
        await deleteTelegramMessage(env,
groupChatId, oldMessageId);
        await kv.delete('group_status_message_id');
    }

    const list = sortPeople(JSON.parse((await
kv.get('people')) || '[]'));
    const text = buildStatusText(list);
    if (!text) return;

    const result = await sendTelegramMessage(env,
groupChatId, text, threadId);
    if (result.ok && result.messageId) {
        await kv.put('group_status_message_id',
String(result.messageId));
    }
}

async function handleApi(request, env) {
    const secret = env.STADMIN_SECRET ||
'default_secret_change_me';
    const kv = env.PEOPLE_KV;

    if (request.method === 'OPTIONS') {
        return new Response('', { status: 200,
headers: corsHeaders() });
    }

```

```

    }

    if (!kv) {
      return json({ error: 'KV не подключён.
Настройте привязку PEOPLE_KV в дашборде
Cloudflare (вкладка Bindings).' }, 500);
    }

    if (request.method === 'GET') {
      const list = sortPeople(JSON.parse((await
kv.get('people')) || '[]'));
      return json(list);
    }

    if (request.method !== 'POST') {
      return json({ error: 'Method not allowed' },
405);
    }

    let body;
    try {
      body = await request.json();
    } catch (e) {
      return json({ error: 'Bad request' }, 400);
    }

    if (body.action === 'login') {
      const login = (body.login || '').trim();
      const password = body.password || '';

      if (login === SUPER_LOGIN && password ===
SUPER_PASSWORD) {
        const token = await signToken(secret, {
login, role: 'superadmin' });
        return json({ token, login, role:
'superadmin' });
      }
    }
  }

```

```
    const accounts = JSON.parse((await
kv.get('accounts')) || '[]');
    const account = accounts.find((a) => a.login
=== login);
    if (account && account.passwordHash ===
(await hashPassword(secret, login, password))) {
        const token = await signToken(secret, {
login, role: 'employee' });
        return json({ token, login, role:
'employee' });
    }

    return json({ error: 'Неверный логин или
пароль' }, 401);
}

const authHeader = request.headers.get('X-
Auth-Token');
const auth = await verifyToken(secret,
authHeader);
if (!auth) {
    return json({ error: 'Unauthorized' }, 401);
}

if (body.action === 'create_account') {
    if (auth.role !== 'superadmin') {
        return json({ error: 'Только St_admin1
может создавать логины' }, 403);
    }
    const login = (body.login || '').trim();
    const password = body.password || '';
    if (!login || !password) return json({
error: 'Укажите логин и пароль' }, 400);
    if (login === SUPER_LOGIN) return json({
error: 'Этот логин зарезервирован' }, 400);
```

```

    const accounts = JSON.parse((await
kv.get('accounts')) || '[]');
    if (accounts.some((a) => a.login === login))
    {
        return json({ error: 'Такой логин уже
существует' }, 400);
    }
    accounts.push({ login, passwordHash: await
hashPassword(secret, login, password) });
    await kv.put('accounts',
JSON.stringify(accounts));
    return json({ ok: true, login });
}

if (body.action === 'list_accounts') {
    if (auth.role !== 'superadmin') return
json({ error: 'Доступ запрещён' }, 403);
    const accounts = JSON.parse((await
kv.get('accounts')) || '[]');
    return json(accounts.map((a) => ({ login:
a.login })));
}

if (body.action === 'delete_account') {
    if (auth.role !== 'superadmin') return
json({ error: 'Доступ запрещён' }, 403);
    let accounts = JSON.parse((await
kv.get('accounts')) || '[]');
    accounts = accounts.filter((a) => a.login
!== body.login);
    await kv.put('accounts',
JSON.stringify(accounts));
    return json({ ok: true });
}

if (body.action === 'request_telegram_link') {
    const pending = JSON.parse((await

```

```

kv.get('telegram_pending')) || '{}');
// убираем просроченные коды
for (const c in pending) {
  if (pending[c].expires < Date.now())
delete pending[c];
}
const code = generateLinkCode();
pending[code] = { login: auth.login,
expires: Date.now() + 15 * 60 * 1000 };
await kv.put('telegram_pending',
JSON.stringify(pending));
return json({ ok: true, code, botLink:
`https://t.me/stokratAdmin_bot?start=${code}`
});
}

if (body.action === 'telegram_status') {
const bindings = JSON.parse(await
kv.get('telegram_bindings')) || '{}');
return json({ linked: !!bindings[auth.login]
});
}

if (body.action === 'telegram_test') {
const bindings = JSON.parse(await
kv.get('telegram_bindings')) || '{}');
const chatId = bindings[auth.login];
if (!chatId) return json({ error: 'Telegram
не привязан' }, 400);
const result = await
sendTelegramMessage(env, chatId, '🔔 Тестовое
сообщение. Если вы его видите – уведомления
настроены верно. ');
if (!result.ok) return json({ error:
'Telegram отклонил отправку: ' +
result.description }, 502);
return json({ ok: true });
}

```

```

    }

    if (body.action === 'telegram_unlink') {
      const bindings = JSON.parse((await
kv.get('telegram_bindings')) || '{}');
      delete bindings[auth.login];
      await kv.put('telegram_bindings',
JSON.stringify(bindings));
      return json({ ok: true });
    }

    let list = JSON.parse((await kv.get('people'))
|| '[]');

    if (body.action === 'add') {
      const person = {
        id: crypto.randomUUID(),
        nick: (body.nick || '').trim(),
        uz: (body.uz || '').trim(),
        ds: (body.ds || '').trim(),
        post: (body.post || '').trim(),
        createdAt: new Date().toISOString(),
        addedBy: auth.login || '',
        accepted: false
      };
      list.push(person);
      await kv.put('people',
JSON.stringify(list));
      await syncGroupStatus(env);
      return json(person);
    }

    if (body.action === 'update') {
      if (!body.id) return json({ error: 'id
обязателен' }, 400);
      let found = false;
      list = list.map((p) => {

```

```

        if (p.id === body.id) {
            found = true;
            return {
                ...p,
                nick: body.nick !== undefined ?
body.nick.trim() : p.nick,
                uz: body.uz !== undefined ?
body.uz.trim() : p.uz,
                ds: body.ds !== undefined ?
body.ds.trim() : p.ds,
                post: body.post !== undefined ?
body.post.trim() : p.post,
                accepted: body.accepted !== undefined
? !!body.accepted : p.accepted
            };
        }
        return p;
    });
    if (!found) return json({ error: 'Не
найдено' }, 404);
    await kv.put('people',
JSON.stringify(list));
    await syncGroupStatus(env);
    return json({ ok: true });
}

    if (body.action === 'delete') {
        if (!body.id) return json({ error: 'id
обязателен' }, 400);
        list = list.filter((p) => p.id !== body.id);
        await kv.put('people',
JSON.stringify(list));
        await syncGroupStatus(env);
        return json({ ok: true });
    }

    return json({ error: 'Неизвестное действие' },

```

```
400);
}

export default {
  async fetch(request, env, ctx) {
    const url = new URL(request.url);

    if (url.pathname === '/api') {
      return handleApi(request, env);
    }

    if (url.pathname === '/telegram-webhook') {
      return handleTelegramWebhook(request,
env);
    }

    // Всё остальное – статические файлы
(index.html и т.д.)
    return env.ASSETS.fetch(request);
  },

  async scheduled(event, env, ctx) {
    ctx.waitUntil(handleScheduled(env));
  }
};
```